

but JavaScript code that acts like a programmable proxy solely responsible for intercepting and responding to network requests. Since it acts as a proxy and can easily be programmed, the application must be served over HTTPS to keep the data secure.

Service worker caches the actual response, including all HTTP headers, rather than just the response data. With this, the application can simply make network requests, and process the response without any specific code to handle the cache.

Tools needed to build a PWA

There is no need to build a PWA from scratch or to convert an existing Web app to a PWA, other than adding a manifest file and service worker configuration. For a basic JavaScript Web application, the details are shown here. We will build an application using the Angular framework and then convert that application into a PWA.

Building a PWA using Angular

In this section we will look at some code examples of how to convert a Web application into a PWA using the very popular Angular framework. There are a few pre-requisites, however.

1. A basic understanding of Angular, Components and Angular schematics is needed. A quick refresher can be found at <https://codelabs.developers.google.com/codelabs/your-first-pwapp/#0>.
2. Angular CLI should be installed on the system in order to run it locally.
3. An HTTP server should be installed on the system to run it locally. Create a new Angular project using the following code:

```
ng new ang-angular-pwa-seed
```

Now go inside the *ang-angular-pwa-seed* directory and run the following command to build the project:

```
ng build --prod
```

Once the project is built, run the following command to invoke the compiled and minified Angular application:

```
http-server -p 8080 -c-1 dist/ang-angular-pwa-seed
```

This will run an HTTP server listening on port 8080 at *http://127.0.0.1:8080/*.

Now, if we run the app in the offline mode, we see the error page. To go offline, right-click on the mouse and click *Inspect*. Then go to the *Application* tab and under *Service Workers* select the *Offline* checkbox.

This will run a bare minimum Angular application without added support for the PWA. Now that we have a basic Angular application setup, we can work on adding the PWA support. To do so, we need to add *@angular/pwa* to the Angular application. Install *@angular/pwa* using the command given below:

```
ng add @angular/pwa
```

The above commands will do the following:

1. Add the *@angular/service-worker* package to your project.
2. Enable the service worker build support in the CLI.
3. Import and register the service worker in the app module.
4. Update the *index.html* file:
 - Include a link to add the *manifest.json* file.
 - Add meta tags for the theme colour.
5. Install icon files to support the installed PWA.
6. Create the service worker configuration file called *ngsw-config.json*.
ngsw-config.json contains the configurations to cache the static assets of the Web application. The contents of the *ngsw-config.json* are

shown below:

```
{
  "$schema": "./node_modules/@angular/
service-worker/config/schema.json",
  "index": "/index.html",
  "assetGroups": [
    {
      "name": "app",
      "installMode": "prefetch",
      "resources": {
        "files": [
          "/favicon.ico",
          "/index.html",
          "/manifest.webmanifest",
          "/*.css",
          "/*.js"
        ]
      }
    },
    {
      "name": "assets",
      "installMode": "lazy",
      "updateMode": "prefetch",
      "resources": {
        "files": [
          "/assets/**",
          "/*.(eot|svg|cur|jpg|png|webp|gif|otf|ttf|woff|woff2|ani)"
        ]
      }
    }
  ]
}
```

Now, run the following command to build the project:

```
ng build --prod
```

Once the project is built, run the following command to invoke the compiled and minified Angular application:

```
http-server -p 8080 -c-1 dist/ang-angular-pwa-seed
```

This will run an HTTP server listening on port 8080 at *http://127.0.0.1:8080/*.

We will try to render the application in the offline mode.